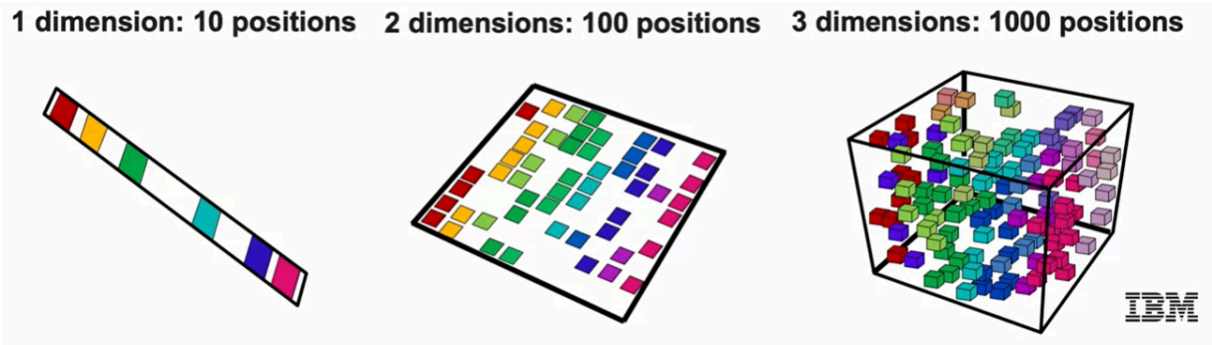
UNSUPERVISED MACHINE LEARNING

MODULE 4:

DIMENSIONALITY REDUCTION

TABLE OF CONTENTS

Introduction to Dimensionality Reduction



Main idea:

- The **curse of dimensionality** means that too many features can harm performance.
- As the number of dimensions increases, the number of observations required grows exponentially, and distance measures become less meaningful.

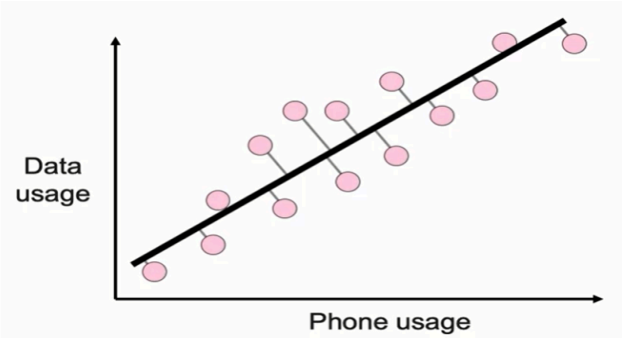
Goal:

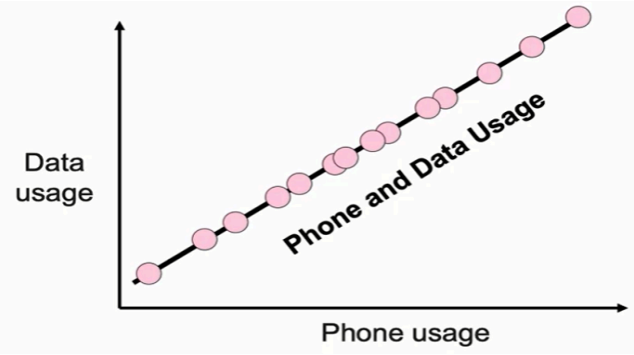
Reduce the number of features while preserving most of the data's useful information.

Two main approaches:

1. **Feature Selection:** Choose a subset of the most important features.
2. **Feature Extraction:** Combine or transform features into new variables (linear or nonlinear).

Principal Component Analysis (PCA)





Concept:

- PCA creates **new features** (called *principal components*) that are **linear combinations** of the original variables.
- These components are **orthogonal** (uncorrelated) and ordered by how much **variance** they explain in the data.

How PCA Works

1. **Projection:**
    - Data points are projected onto new axes that maximize variance.
    - The first component captures the most variance; the second captures the next most (and is orthogonal to the first), and so on.

2. **Mathematical foundation — Singular Value Decomposition (SVD):**

$$A = USV^T$$

    - A: Original data matrix (size m×n)
    - U, V: Orthogonal matrices (rotations)
    - S: Diagonal matrix with singular values (lengths of principal components)
    - Larger singular values = more important principal components

3. **Dimensionality reduction:**

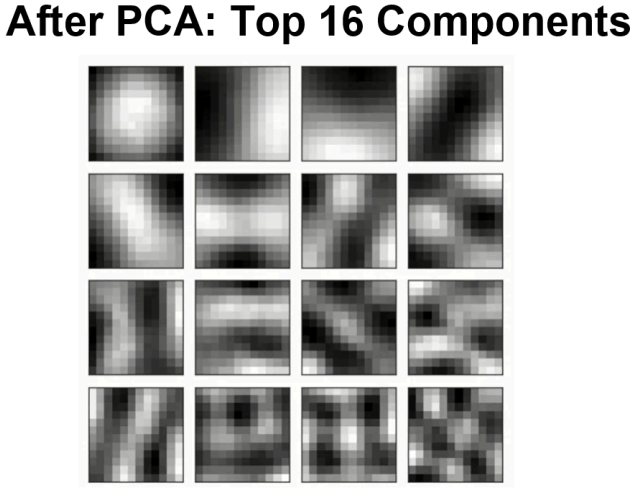
$$A_{(m\times n)} \times V^T_{(n\times k)} = A'_{(m\times k)}$$

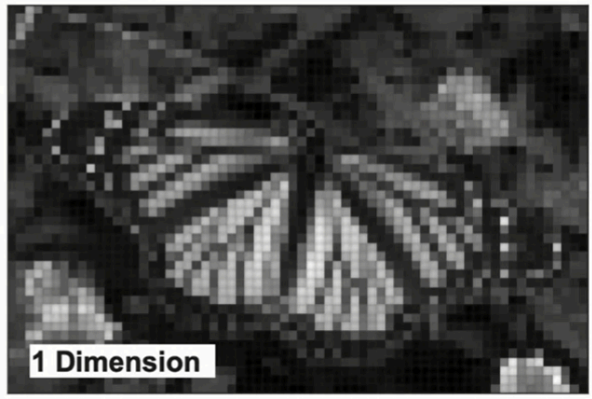
→ Keeps the same number of rows but reduces features from *n* to *k*.

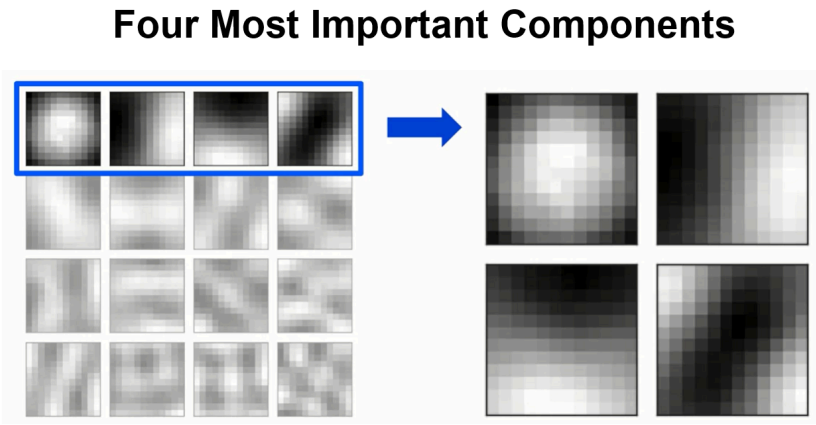
4. **Data scaling:**
    - PCA is sensitive to scale, since variance is affected by measurement units.
    - Always standardize data before applying PCA.

```
from sklearn.decomposition import PCA

# Create PCA instance with desired number of components
pca = PCA(n_components=3)

# Fit and transform training data
X_transformed = pca.fit_transform(X_train)
```

Example of PCA:







PCA in Image:

```
from sklearn.decomposition import PCA

pca = PCA(n_components=16)
compressed = pca.fit_transform(image_matrix)
reconstructed = pca.inverse_transform(compressed)
```

Real-world Applications of PCA

- **Image Compression:** Reduce pixel data while keeping visual quality.
- **Text/NLP:** Reduce word-count feature space.
- **Customer Analytics:** Compress correlated behavioral features.
- **Finance:** Identify hidden factors affecting stock movements.

Key Takeaways

- **Dimensionality Reduction** combats the curse of dimensionality by removing redundancy.
- **PCA**:
    - Projects data into orthogonal axes (principal components).
    - Keeps the most variance with fewer dimensions.
    - Relies on **SVD decomposition** and **scaling** for accuracy.
- In practice:
    - Use PCA before clustering or visualization.
    - Choose the number of components based on explained variance.